Agent Orchestrator – Executive Overview

Agent Orchestrator Team

2026-04-01

Agent Orchestrator

Executive Overview

Version 1.0

April 1, 2026

Table of contents

1 Executive Summary **2**

2 Problem Statement **3**

3 Platform Architecture **3**

4 Key Capabilities **4**

5 Configuration-Driven Design **5**

6 Governance Framework **6**
- 6.1 Non-Blocking Policy Evaluation . . . 6
- 6.2 Delegated Authority . . . 6
- 6.3 Hash-Chained Audit Trail . . . 6

7 Connector Ecosystem **7**
- 7.1 Capability Types . . . 7
- 7.2 MCP Bridge . . . 7
- 7.3 Provider Extension . . . 7

8 Evaluation and Quality **7**
- 8.1 LLM-as-Judge . . . 7
- 8.2 A/B Testing . . . 8
- 8.3 Benchmarks . . . 8
- 8.4 Simulation Sandbox . . . 8

9 Integration Points **8**

10 Technology Stack **8**

11 Deployment Options **9**
- 11.1 Lite Mode . . . 9
- 11.2 Standard Mode . . . 9
- 11.3 Enterprise Mode . . . 9

12 API Surface **9**

13 Roadmap **10**

14 Summary **11**

1 Executive Summary

Agent Orchestrator is a domain-agnostic platform for orchestrating multi-agent AI workflows with built-in governance, auditing, and observability. The platform coordinates multiple AI agents through configurable phase-based pipelines, with all domain knowledge expressed in YAML configuration profiles. Zero domain-specific logic is hardcoded into the platform.

This architecture means a single platform installation can power content moderation, software development, security incident response, research coordination, or any other multi-agent workflow – simply by loading a different YAML profile. The platform provides the orchestration engine, governance framework, connector ecosystem, evaluation tooling, and audit infrastructure that every multi-agent system needs, while leaving domain decisions entirely to configuration.

The platform supports five LLM providers (OpenAI, Anthropic, Google, Grok, Ollama), exposes 40+ REST API endpoints, includes a full CLI, and deploys via Docker with PostgreSQL for production use.

2 Problem Statement

Organizations building multi-agent AI systems face four recurring challenges:

Agent Coordination Complexity. Orchestrating multiple AI agents through structured workflows requires managing concurrency, sequencing, failure handling, retries, and state propagation. Without a dedicated orchestration layer, teams build brittle, one-off coordination logic that is difficult to test, modify, and scale.

Governance Gaps. Multi-agent systems make decisions at machine speed, but organizations need policy enforcement, approval workflows, and delegated authority. Most agent frameworks provide no governance layer, forcing teams to bolt on ad-hoc approval processes after deployment.

Knowledge Loss. Agent outputs, intermediate results, and decision rationale are typically discarded after execution. When issues arise, there is no way to reconstruct what happened, why decisions were made, or what data informed them. Evaluation and optimization are impossible without historical output data.

Observability Blind Spots. When a multi-agent pipeline produces poor results, debugging requires understanding which agent, in which phase, with what input, produced what output. Standard application monitoring tools are insufficient because they lack awareness of agent-level concepts like phases, governance decisions, and connector invocations.

Agent Orchestrator addresses all four challenges through a unified, domain-agnostic platform.

3 Platform Architecture

The platform is organized in a layered architecture with clear separation of concerns:

```
REST API (FastAPI) + CLI (Click)
        |
Core Engine
  - OrchestrationEngine    Lifecycle controller
  - PipelineManager         Phase graph traversal
  - WorkQueue               Priority-ordered async queue
  - AgentPool               Concurrency-limited agent execution
  - PhaseExecutor           Parallel/sequential agent execution
  - AgentExecutor           Single-agent LLM call with retry
  - EventBus                Async pub/sub typed events
        |
```

```
Governance Layer
  - Governor                Non-blocking policy evaluation
  - AuditLogger             Hash-chained JSONL audit trail
  - ReviewQueue             Human review queue for escalations
        |
Connector Framework
  - ConnectorService        Capability-based external system access
  - ConnectorExecutor       Timeout, backoff, cost tracking
  - ConnectorRegistry       Provider and config storage
  - ConnectorGovernance     Permission enforcement
  - MCP Bridge              Model Context Protocol integration
        |
Adapters
  - LLMAdapter              Multi-provider LLM routing (5 providers)
  - MetricsCollector        Execution metrics persistence
  - WebhookAdapter          Outbound event notifications
        |
Persistence
  - SettingsStore            Configuration storage
  - StateStore               Runtime state per work item
  - ConfigHistory            Versioned config with rollback
```

Data flow: A work item is submitted via API or CLI. The `OrchestrationEngine` places it in the `WorkQueue`. The processing loop dequeues work items by priority, evaluates governance policies via the `Governor`, and hands execution to the `PipelineManager`. The `PipelineManager` traverses the phase graph, delegating each phase to the `PhaseExecutor`, which runs agents through the `AgentPool`. Each agent call goes through the `LLMAdapter` to the configured LLM provider. Every step is recorded in the hash-chained audit trail.

4 Key Capabilities

Capability	Description	Key Components
Orchestration	Phase-based pipelines with parallel/sequential execution, priority queuing, pause/resume	OrchestrationEngine, PipelineManager, WorkQueue, AgentPool
Multi-Provider LLM	5 providers with per-agent model selection, auto-registration based on available keys	LLMAdapter (OpenAI, Anthropic, Google, Grok, Ollama)
Governance	Non-blocking policy evaluation, delegated authority with auto-approve/escalate/review tiers	Governor, ReviewQueue, PolicyConfig

Capability	Description	Key Components
Audit	Tamper-evident hash-chained JSONL log covering all decisions, actions, and transitions	AuditLogger with SHA-256 chain, 7 record types
Knowledge	Content-addressable artifact storage with semantic search and auto-extraction	Knowledge Store, embedding-based retrieval
Connectors	10 capability types, 12+ providers, MCP bridge, deny-by-default permissions	ConnectorService, ConnectorGovernance, MCP Client/Server
Evaluation	LLM-as-judge quality assessment, A/B testing, benchmarks, simulation sandbox	Evaluation framework, experiment management
Observability	Structured metrics, connector tracing, event bus, webhook notifications	MetricsCollector, ConnectorTraceStore, EventBus

5 Configuration-Driven Design

All domain knowledge lives in four YAML files within a profile directory. The platform core interprets these files without any awareness of what domain they represent.

File	Purpose	Contains
`agents.yaml`	Agent definitions	Identity, system prompts, LLM config, skills, concurrency, retry policies, connector permissions
`workflow.yaml`	Phase pipeline	Phase graph with agents, parallel/sequential execution, success/failure transitions, quality gates, terminal phases
`governance.yaml`	Policy rules	Delegated authority thresholds, policy definitions (threshold, keyword, pattern rules), escalation actions
`settings.yaml`	Infrastructure	API keys, LLM endpoints, deployment mode, persistence backend

Profile switching is a runtime operation. An organization can maintain separate profiles for development, staging, and production, or for entirely different use cases, and switch between them without code changes or restarts.

Configuration hot-reload allows updates to agents, workflows, and policies without engine restart. Changes are versioned with `ConfigHistory` for audit and rollback.

Built-in templates provide starting points:

Template	Use Case	Agents	Phases
`content-moderation`	Content classification and policy enforcement	Classifier, Policy Evaluator, Escalation Handler	classify, evaluate, decide, escalate
`software-dev`	Requirements to code with review	Requirements Analyst, Code Generator, Code Reviewer	analyze, generate, review, revise
`research-team`	Literature search and synthesis	Research Scout, Synthesizer, Critical Evaluator	search, synthesize, evaluate

6 Governance Framework

6.1 Non-Blocking Policy Evaluation

The `Governor` evaluates policies against work item data and agent outputs without blocking the processing pipeline. Governance is designed to run in parallel with execution, logging decisions to the audit trail.

6.2 Delegated Authority

A three-tier authority model balances automation speed with human oversight:

Tier	Confidence	Behavior
Auto-approve	>= 0.8 (configurable)	Proceeds without intervention
Escalate	>= 0.5 and < 0.8	Flagged and logged, processing continues
Human review	< 0.5	Queued for human approval, processing pauses

Thresholds are configurable per profile, allowing organizations to tighten or relax oversight based on risk tolerance.

6.3 Hash-Chained Audit Trail

Every governance decision, phase transition, agent action, work item change, and connector invocation is logged in an append-only JSONL file with SHA-256 hash chaining. Each record's hash includes the previous record's hash, creating a tamper-evident chain.

The audit trail records seven event types: `GOVERNANCE_DECISION`, `PHASE_TRANSITION`, `AGENT_ACTION`, `SYSTEM_EVENT`, `WORK_ITEM_CHANGE`, `CONNECTOR_INVOCATION`, and `MCP_INVOCATION`.

7 Connector Ecosystem

7.1 Capability Types

The connector framework abstracts external system access through 10 capability types. Agents request a capability, not a specific provider, enabling provider substitution without configuration changes.

Capability Type	Description	Built-in Providers
`search`	Full-text and semantic search	Tavily, SerpAPI, Brave
`documents`	Document retrieval and management	Confluence
`messaging`	Send/receive messages	Slack, Teams, Email
`ticketing`	Issue and ticket management	Jira, Linear
`repository`	Source code repositories	GitHub, GitLab
`telemetry`	Metrics, logs, traces	(extension point)
`identity`	User and group lookup	(extension point)
`external_api`	Generic HTTP/REST endpoints	(extension point)
`file_storage`	Object and blob storage	(extension point)
`workflow_action`	Trigger external workflow steps	(extension point)

7.2 MCP Bridge

Model Context Protocol (MCP) integration provides bidirectional AI ecosystem interoperability:

- **MCP Client:** Agents consume tools from external MCP servers (GitHub, Slack, filesystem, custom servers)
- **MCP Server:** Platform exposes governed capabilities to external AI clients (Claude Desktop, Cursor, other MCP-compatible tools)

All MCP invocations flow through the existing governance pipeline, receiving permission checks, contract validation, and audit logging.

7.3 Provider Extension

New providers use structural typing (Python Protocol) – no inheritance required. Any class implementing two methods (`get_descriptor` and `execute`) is a valid provider. Providers can be discovered automatically from plugin directories or Python entry points.

8 Evaluation and Quality

8.1 LLM-as-Judge

A designated evaluator LLM assesses agent output quality against configurable criteria (accuracy, completeness, clarity, relevance). Evaluation results feed back into governance decisions and are

stored for longitudinal analysis.

8.2 A/B Testing

The platform supports controlled experiments comparing different agent configurations, LLM models, or prompt variations. Work items are routed to control and variant groups, with quality metrics collected per variant for statistical comparison.

8.3 Benchmarks

Curated datasets with known-good outputs measure agent performance across quality scores, latency, token usage, and cost. Benchmarks establish baselines and detect regressions when configurations change.

8.4 Simulation Sandbox

A sandbox mode intercepts LLM and connector calls with configurable mocks while running the full governance and audit pipeline. This enables workflow validation, policy testing, and cost estimation without incurring real API costs.

9 Integration Points

Agent Orchestrator is designed as the runtime engine within a broader AI development ecosystem:

System	Integration	Purpose
AO Studio	Visual designer that exports YAML profiles	Design agent workflows visually, generate configuration files, monitor execution
Auto-Architect	Code generation platform that produces agent configurations	Generate agents.yaml and workflow.yaml from natural language requirements
SICS	Security Incident Command System built on Agent Orchestrator	Domain-specific security profile with Red/Blue/White team agents for incident response

Each integration consumes or produces the same YAML configuration profiles, maintaining the platform's domain-agnostic core while enabling specialized tooling.

10 Technology Stack

Layer	Technology	Version
Language	Python	3.11+
API Framework	FastAPI	0.100+

Layer	Technology	Version
Data Validation	Pydantic	v2
CLI Framework	Click	8.x
Database	PostgreSQL	16
Container Runtime	Docker + Docker Compose	Latest
Local LLM	Ollama	Latest
LLM SDKs	openai, anthropic, google-generativeai	Latest
MCP	mcp (optional)	Latest
Audit Storage	JSONL with SHA-256 hash chain	–
Metrics Storage	JSONL	–
Configuration	YAML	–
Testing	pytest	871+ tests passing

11 Deployment Options

11.1 Lite Mode

Single-process deployment with file-based or SQLite storage. No external dependencies beyond Python 3.10+ and at least one LLM API key. Installed via `pip install agent-orchestrator`.

Best for: solo developers, prototyping, CI/CD pipeline agents, demos.

11.2 Standard Mode

Docker Compose deployment with PostgreSQL for durable state and audit trails. Three containers: API server (port 8000), PostgreSQL (port 5432), and optional Ollama (port 11434).

Best for: production teams, multi-service environments.

11.3 Enterprise Mode

Docker Compose with PostgreSQL plus an authentication gateway. Adds multi-tenant workspace isolation, RBAC, and quota management.

Best for: large organizations, regulated industries, multi-tenant deployments.

12 API Surface

The platform exposes 40+ REST API endpoints organized in 20 route groups under `/api/v1`:

Group	Prefix	Count	Description
Health	`/health`	3	Liveness, readiness, general health
Context	`/context`	1	Execution context
Agents	`/agents`	7	Full CRUD, import/export
Workflow	`/workflow`	3	Phase listing and detail
Work Items	`/workitems`	4	Submit, list, get, cancel
Governance	`/governance`	7	Policy CRUD, review queue
Execution	`/execution`	5	Start, stop, pause, resume, status

Group	Prefix	Count	Description
Metrics	`/metrics`	2	Execution metrics
Audit	`/audit`	3	Trail query, detail, chain verify
Config	`/config`	4	Validation, settings, profiles
Connectors	`/connectors`	7	Capabilities, providers, execution, traces
Contracts	`/contracts`	5	Capability and artifact contracts
Knowledge	`/knowledge`	4	Store, retrieve, search, delete
Evaluation	`/evaluation`	3	Run evaluations, results, criteria
Experiments	`/experiments`	4	A/B test management
Benchmarks	`/benchmarks`	3	Run, results, datasets
Simulation	`/simulation`	2	Sandbox creation and execution
Events	`/events`	2	Subscription and history
Webhooks	`/webhooks`	3	Register, list, delete
MCP	`/mcp`	2	Server status, tool listing

Interactive API documentation: `http://localhost:8000/docs` (Swagger UI) and `http://localhost:8000/redoc` (ReDoc).

13 Roadmap

Priority	Capability	Description
Near-term	Database-backed audit	Move audit trail from JSONL to PostgreSQL for improved query performance
Near-term	Audit log rotation	Automatic archival and rotation of audit log files
Near-term	Human review resolution	Complete the review queue with approval/rejection workflows
Mid-term	Agent versioning	Non-destructive agent definition updates with version history
Mid-term	Conditional branching	Phase transitions based on output content, not just success/failure
Mid-term	Phase-level timeouts	Wallclock timeout enforcement at the phase level
Mid-term	SIEM export	Audit trail export to Splunk, Datadog, and other SIEM systems
Long-term	RBAC	Role-based access control for multi-tenant deployments
Long-term	Agent health monitoring	Heartbeat and readiness probes for long-running agents
Long-term	Cyclic phase graphs	Support for explicit loops and re-entry in phase pipelines

14 Summary

Agent Orchestrator provides a complete, domain-agnostic runtime for multi-agent AI systems. Its configuration-driven architecture ensures that all domain knowledge lives in portable YAML profiles while the platform handles orchestration, governance, auditing, external system integration, and evaluation.

The platform is production-ready for core orchestration, governance, and audit capabilities, with 871+ tests passing and 11 built-in connector providers. Active development focuses on expanding the evaluation framework, knowledge store, and enterprise deployment features.

Organizations adopting Agent Orchestrator gain a single platform that can power any multi-agent workflow, with the governance and observability infrastructure required for production deployment, and the evaluation tooling needed for continuous improvement.